gc_content.py 사용 설명서

1. 개요

lab11/src/gc_content.py 는 FASTQ 파일의 각 read 마다 GC 비율과 평균 품질을 함께 계산한다. 결과는 이름<TAB>GC 비율<TAB>평균 품질 형식으로 read 하나당 한 줄씩 출력한다. FastQC 의 Per sequence GC content 와 Per sequence quality scores 그래프가 read 마다 계산하는 값과 같다.

마지막에는 GC 비율이 가장 낮은 read 10 개와 가장 높은 read 10 개를 골라, 각 묶음의 평균 품질이 전체 read 평균과 얼마나 다른지 화면(stderr)에 함께 보여 준다.

2. 실행 방법

```
cd lab11
python3 src/gc_content.py <FASTQ 파일>
python3 src/gc_content.py <FASTQ 파일> > 결과.tsv
```

- 인자는 FASTQ 파일 경로 하나만 받는다.
- read 수만큼 줄이 나오므로 보통 > 로 파일에 저장한다. 같은 이름의 파일이 있으면 덮어쓰므로 먼저 확인한다.
- 압축 파일(.fq.gz)은 바로 읽지 못한다. 먼저 gunzip -k 로 푼다.
- 외부 라이브러리 없이 파이썬 표준 기능만 쓴다.
- 품질 문자는 Phred+33 인코딩(현재 Illumina 표준)이라고 가정한다.

3. 출력 형식

첫 줄은 머리줄 name gc mean_qual 이고, 그 아래로 read 하나당 한 줄씩 나온다. 열 사이는 탭으로 구분한다.

열	이름	내용
1열	name	read 이름 (헤더 @ 뒤의 첫 단어)
2열	gc	GC 비율, 0 ~ 1 (소수점 넷째 자리까지 반올림)
3열	mean_qual	평균 Phred 품질 (소수점 둘째 자리까지 반올림)

- GC 비율은 lab07 fasta.py 의 gc_content 와 같은 기준이다: (G 수 + C 수) / 서열 길이.
- 퍼센트(%)로 보려면 100 을 곱한다. 예: 0.4257 → 42.57%.

출력이 두 갈래로 나뉜다

출력	내용	> 결과.tsv 로 저장하면
stdout (표준 출력)	read 별 TSV 결과	파일로 들어감
stderr (표준 오류)	GC 극단 read 비교 결과	파일에 들어가지 않고 화면에 남음

- 그래서 TSV 파일에는 머리줄과 read 별 3 열만 들어간다.
- 비교 결과만 보고 TSV 는 버리려면: python3 src/gc_content.py <FASTQ> > /dev/null
- 비교 결과도 파일로 남기려면: python3 src/gc_content.py <FASTQ> > 결과.tsv 2> 비교.txt

4. 계산 방법: 반복문 안의 반복문

```
바깥 반복: read 하나씩                → read 개수(N) 만큼
   안쪽 반복 1: 서열 글자 하나씩      → read 길이만큼
        G 나 C 면 gc += 1
   안쪽 반복 2: 품질 글자 하나씩      → read 길이만큼
        total += ord(ch) - 33
   GC 비율 = gc / 길이,  평균 품질 = total / 길이
   한 줄 출력
```

- 전체 반복 횟수는 대략 전체 염기 수 x 2 이다 (서열 한 번, 품질 한 번).
- 결과가 read 마다 바로 출력되므로 qual_by_position.py 처럼 위치별 칸을 모아 둘 필요가 없다. read 길이가 달라도 각 read 의 길이로 나누므로 문제없다.

손으로 계산한 예 (소문자, N, 길이가 다른 read 포함):

read	서열	품질	GC 비율	평균 품질
r1	ACGT	IIHH	2/4 = 0.5	(40+40+39+39)/4 = 39.5
r2	ggCA	I#I#	3/4 = 0.75	(40+2+40+2)/4 = 21.0
r3	AAAN	####	0/4 = 0.0	2.0
r4	GCGCG	IIIII	5/5 = 1.0	40.0

```
$ python3 src/gc_content.py gc_small.fastq 2> /dev/null    # 비교 결과(stderr)는 숨김
name    gc      mean_qual
r1      0.5     39.5
r2      0.75    21.0
r3      0.0     2.0
r4      1.0     40.0
```

5. 실행 예시 (lab11 데이터)

```
$ python3 src/gc_content.py data/SRR1553607_1.fastq > gc_srr1.tsv
$ head -4 gc_srr1.tsv
name    gc      mean_qual
SRR1553607.1    0.4257  37.44
SRR1553607.2    0.4653  36.62
SRR1553607.3    0.6733  31.74
```

- read 203,445 개 → 머리줄 포함 203,446 줄. 실행 시간 약 2.2 초.
- 이때 GC 극단 read 비교 결과는 파일이 아니라 화면에 나온다 (7장).
- data/trimmed.fastq 는 read 188,357 개 → 188,358 줄.

6. 검증

비교 대상	방법	결과
손 계산	위 4장 샘플	모두 일치
awk 로 따로 계산	두 파일의 모든 read 를 한 줄씩 비교	이름 모두 일치, 차이는 반올림 자릿수 이내
FastQC %GC	Basic Statistics 의 %GC = 58	read GC 평균 0.5884 (FastQC 는 정수로 버림)
FastQC Per sequence quality scores	read 평균 품질을 정수로 버림해 개수를 셈	40 개 구간 모두 일치
GC 극단 read 선택	awk 결과를 sort 로 따로 정렬해 최저·최고 10 개 GC 값 비교	두 파일 모두 일치
stdout / stderr 분리	> 결과.tsv 로 저장한 파일에 비교 문구가 있는지	0 줄 (섞이지 않음)

FastQC 결과는 아래 명령으로 볼 수 있다.

```
unzip -p doc/SRR1553607_1_fastqc.zip '*/fastqc_data.txt' \
  | sed -n '/>>Per sequence quality scores/,/>>END_MODULE/p'
```

7. GC 극단 read 와 평균 품질 비교

방법

1. read 마다 구한 (이름, GC 비율, 평균 품질) 을 results 리스트에 모은다.
2. GC 비율 순으로 정렬해 앞 10 개(GC 최저)와 뒤 10 개(GC 최고)를 고른다.
3. 각 묶음의 평균 품질과 전체 read 평균 품질(read 평균들의 평균)의 차이를 출력한다.

SRR1553607_1.fastq 결과 (원본)

```
$ python3 src/gc_content.py data/SRR1553607_1.fastq > /dev/null

== GC 극단 read 와 전체 평균 품질 비교 (read 203445 개) ==

[GC 가 가장 낮은 10 개]
name    gc      mean_qual
SRR1553607.166164       0.1287  39.66
SRR1553607.165570       0.1386  38.32
SRR1553607.179986       0.1386  38.09
SRR1553607.59968        0.1584  39.58
SRR1553607.61985        0.1584  33.28
SRR1553607.70984        0.1584  39.22
SRR1553607.117728       0.1584  39.27
SRR1553607.117814       0.1584  37.28
SRR1553607.166721       0.1584  40.11
SRR1553607.172662       0.1584  38.46
묶음 평균 품질: 38.33  (전체 평균 대비 +4.88)

[GC 가 가장 높은 10 개]
name    gc      mean_qual
SRR1553607.53820        0.9802  11.1
SRR1553607.169331       0.9505  19.73
SRR1553607.143780       0.9406  9.25
SRR1553607.77585        0.9406  9.06
```

```
SRR1553607.194620       0.9307  17.33
SRR1553607.169682       0.9307  32.28
SRR1553607.106561       0.9307  24.01
SRR1553607.195126       0.9208  34.17
SRR1553607.185657       0.9208  10.21
SRR1553607.176568       0.9208  5.75
묶음 평균 품질: 17.29  (전체 평균 대비 -16.16)

전체 read 평균 품질: 33.45
```

두 파일 요약

파일	전체 평균	GC 최저 10 개	차이	GC 최고 10 개	차이
SRR1553607_1 (원본)	33.45	38.33	+4.88	17.29	-16.16
trimmed (fastp 후)	34.95	38.46	+3.51	32.02	-2.93

해석

- GC 가 아주 높은 read 는 품질이 크게 낮다. 원본에서 GC 최고 10 개의 평균 품질은 17.29 로 Q20 에도 못 미친다. 전체 read 평균 품질의 표준편차가 6.36 이므로 -16.16 은 표준편차의 2.5 배 정도 되는 큰 차이다. Illumina 시퀀싱은 GC 가 높은 구간에서 오류가 늘어나는 경향이 있다.
- GC 가 낮은 read 는 오히려 품질이 높다 (+4.88).
- 10 개만의 우연이 아니다. GC 를 0.1 단위 구간으로 나눠 모든 read 를 보면 같은 경향이 나온다.

GC 구간	원본 read 수	원본 평균 품질	trimmed read 수	trimmed 평균 품질
0.1-0.2	54	37.61	72	38.04
0.2-0.3	1,559	37.59	1,722	37.82
0.3-0.4	8,998	36.61	8,903	37.13
0.4-0.5	27,096	33.53	23,332	35.43
0.5-0.6	61,447	33.84	56,176	35.22
0.6-0.7	79,766	33.38	75,953	34.64
0.7-0.8	23,491	31.41	21,434	33.74
0.8-0.9	1,007	26.66	738	32.81
0.9-1.0	27	20.01	27	31.23

- fastp 가 고GC 저품질 read 를 많이 걸러 냈다. 원본 GC 최고 10 개 중 trimmed 에 남은 것은 3 개뿐이다 (품질 24.01, 32.28, 34.17 인 read). 그래서 trimmed 에서는 차이가 -2.93 으로 줄었다. trimming 으로 read 가 짧아지면 GC 비율도 바뀌므로 trimmed 의 극단 read 는 원본과 다른 read 이다.

주의: 동점(tie)

GC 비율은 (GC 수) / (길이) 이므로 길이가 101 로 같으면 같은 값이 많이 나온다. 10 번째 자리에 같은 GC 를 가진 read 가 여러 개면, 어느 read 가 뽑힐지는 파일 순서에 따라 정해진다 (sorted 는 같은 값의 순서를 그대로 둔다).

파일 / 묶음	경계 GC	확정된 read	동점 read	묶음 평균이 될 수 있는 범위
원본 / 최저	0.1584	3	7 (7 자리)	동점 없음, 38.33 고정
원본 / 최고	0.9208	7	6 (3 자리)	14.47 ~ 21.59
trimmed / 최저	0.1584	9	7 (1 자리)	37.83 ~ 38.51
trimmed / 최고	0.9307	8	2 (2 자리)	동점 없음, 32.02 고정

어떻게 골라도 "GC 최고 묶음은 전체 평균보다 훨씬 낮고, GC 최저 묶음은 더 높다" 는 결론은 바뀌지 않는다. 다만 read 10 개는 표본이 작으므로 구간별 표처럼 전체 read 를 함께 보는 것이 좋다.

8. 코드 설명

read_fastq(filename)

파일을 4줄씩 끊어서 (이름, 서열, 품질) 튜플의 리스트를 돌려준다. 이름은 헤더의 첫 단어만 쓴다 (@SRR1553607.1 1 length=101 → SRR1553607.1).

gc_ratio(seq)

1. 서열이 비어 있으면 0.0 을 돌려준다.
2. seq.upper() 로 대문자로 바꾼 뒤 글자를 하나씩 보며 G 나 C 면 1 을 더한다.
3. gc / len(seq) 를 돌려준다.

lab07 의 gc_content 는 대문자만 세지만, 이 함수는 소문자(soft-masking)도 센다. N 은 길이에는 들어가고 GC 에는 들어가지 않는다.

mean_quality(qual)

품질 글자마다 ord(ch) - 33 을 더해 길이로 나눈다. 빈 문자열이면 0.0 을 돌려준다.

average(values)

리스트의 합을 개수로 나눈다.

report_extremes(results, n=10)

1. sorted(results, key=lambda r: r[1]) 로 GC 비율(튜플의 두 번째 값) 오름차순 정렬한다.
2. 앞에서 n 개를 GC 최저 묶음, 거꾸로 뒤집어 앞에서 n 개를 GC 최고 묶음으로 쓴다.
3. 전체 평균 품질과 각 묶음의 평균 품질, 그 차이를 print(..., file=sys.stderr) 로 출력한다.
4. read 가 2 x n 개보다 적으면 두 묶음이 겹친다는 주의 문구를 붙인다.

n 을 바꾸려면 마지막 줄을 report_extremes(results, 50) 처럼 고친다.

메인 부분

1. sys.argv[1] 의 파일을 read_fastq 로 읽는다.
2. 머리줄을 출력한다.
3. read 마다 gc_ratio, mean_quality 를 구해 탭으로 이어 출력하고, results 에도 모은다.
4. read 가 하나 이상이면 report_extremes 로 비교 결과를 출력한다.

9. 결과 활용 예

```
# GC 비율이 0.6 이상인 read 수
awk 'NR>1 && $2>=0.6' gc_srr1.tsv | wc -l

# 평균 품질이 20 미만인 read 이름
awk 'NR>1 && $3<20 {print $1}' gc_srr1.tsv | head
```

10. 오류가 나는 경우

상황	결과
파일 이름을 주지 않고 실행	IndexError: list index out of range
빈 파일	오류 없이 머리줄만 출력, 비교 결과는 출력하지 않음
read 가 20 개보다 적음	두 묶음이 겹치고 "주의" 문구가 나옴 (4 개면 두 묶음 모두 전체와 같음)
헤더 줄이 @ 뿐이고 이름이 없음	split()[0] 에서 IndexError
파일 끝에 빈 줄이 더 있음	4줄 묶음이 어긋나 IndexError 가 나거나 값이 틀어짐
서열이 여러 줄로 나뉜 FASTQ	4줄 고정이라고 가정하므로 잘못 읽음
Phred+64 인코딩(오래된 Illumina)	오류는 없지만 품질이 31 씩 높게 나옴
아주 큰 파일	readlines() 가 파일 전체를 메모리에 올리므로 느리거나 메모리가 부족할 수 있음

- 서열과 품질 문자열의 길이가 같은지는 확인하지 않는다.
- N 이 많은 read 는 GC 비율이 낮게 나온다. N 을 빼고 계산하려면 분모를 len(seq) - seq.count("N") 으로 바꾼다.

부록. GC 구간별 평균 품질 그래프

lab11 데이터 두 개에 gc_content.py 를 실행해 얻은 read 별 결과를 GC 비율 0.1 구간으로 묶고, 구간마다 read 평균 품질의 평균을 그렸다. 점선은 각 파일의 전체 read 평균 품질이다. 배경 띠는 FastQC 와 같은 기준(Q28 이상, Q20~28, Q20 미만)이다.

